ЗВІТ з практичної роботи № 16

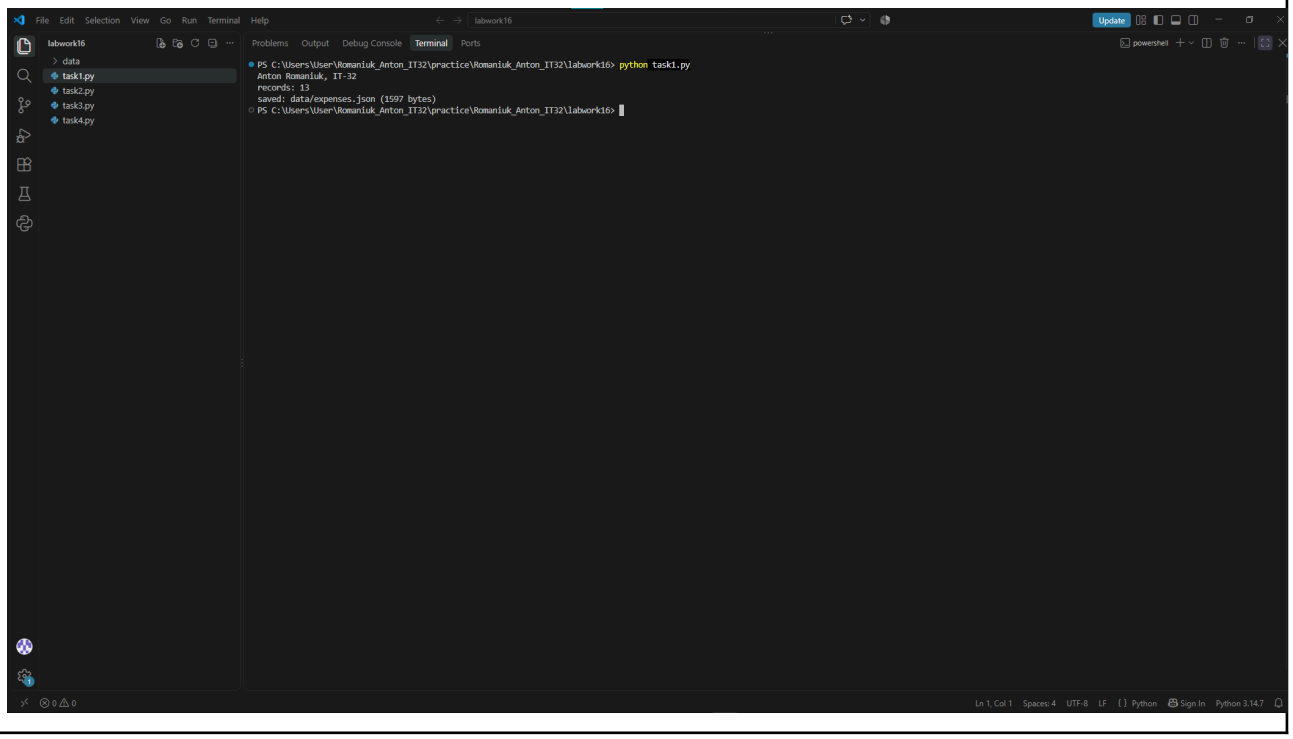
Студент: Anton Romaniuk

Група: IT-32

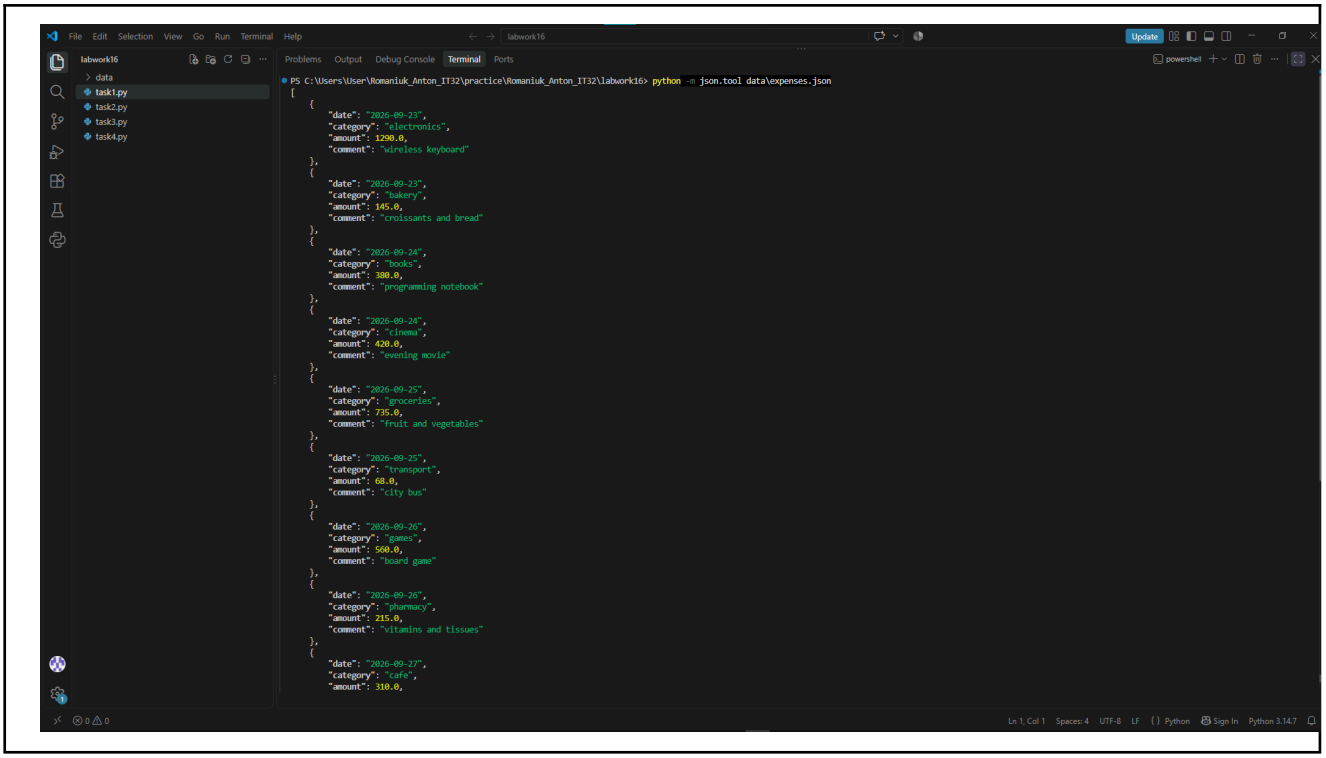
Завдання 1. CSV → JSON

Програма task1.py перетворює data/expenses.csv на data/expenses.json: список об'єктів з полями date, category, amount (число) і comment. Файл записується з параметрами indent=2, ensure_ascii=False і кодуванням utf-8; програма виводить кількість записаних об'єктів та розмір файлу в байтах. Рядок-заголовок CSV (date;category;amount;note) пропускається.

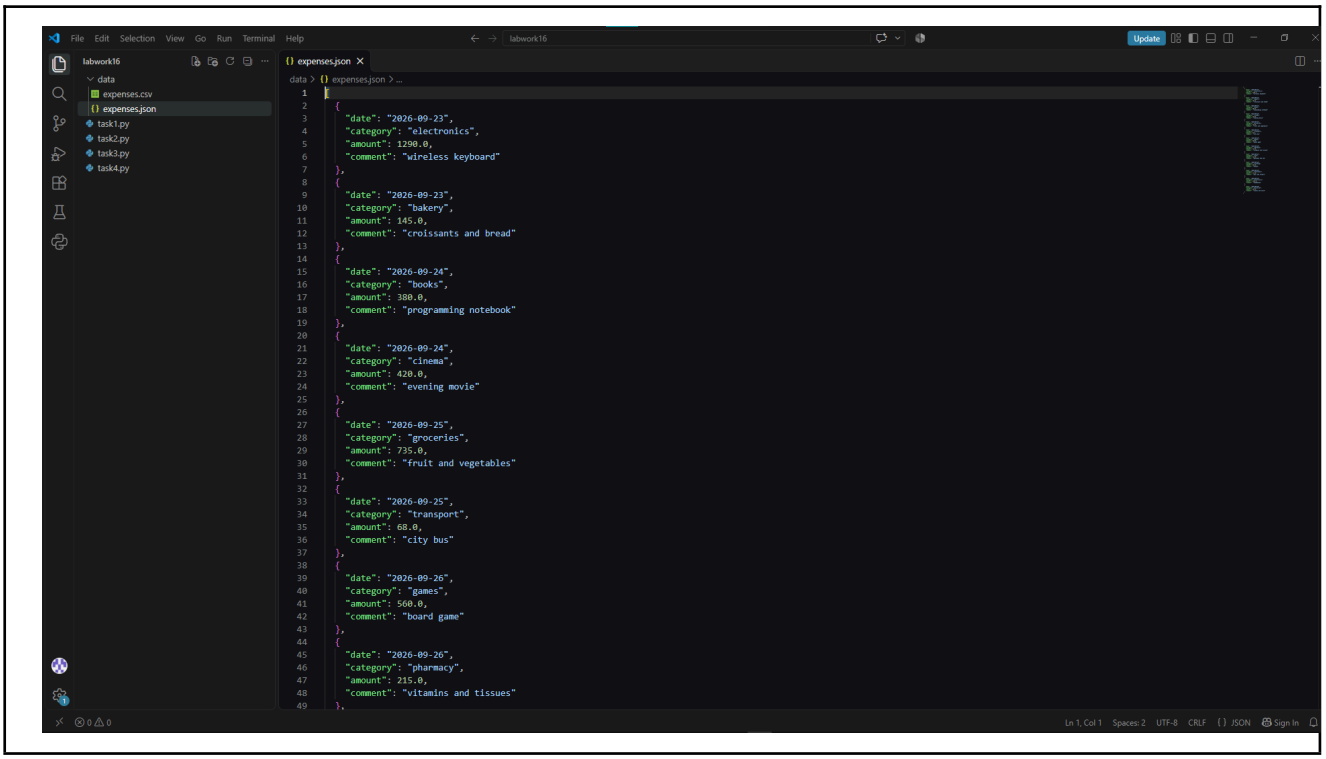
```
python task1.py
```




```
python -m json.tool data\expenses.json
```



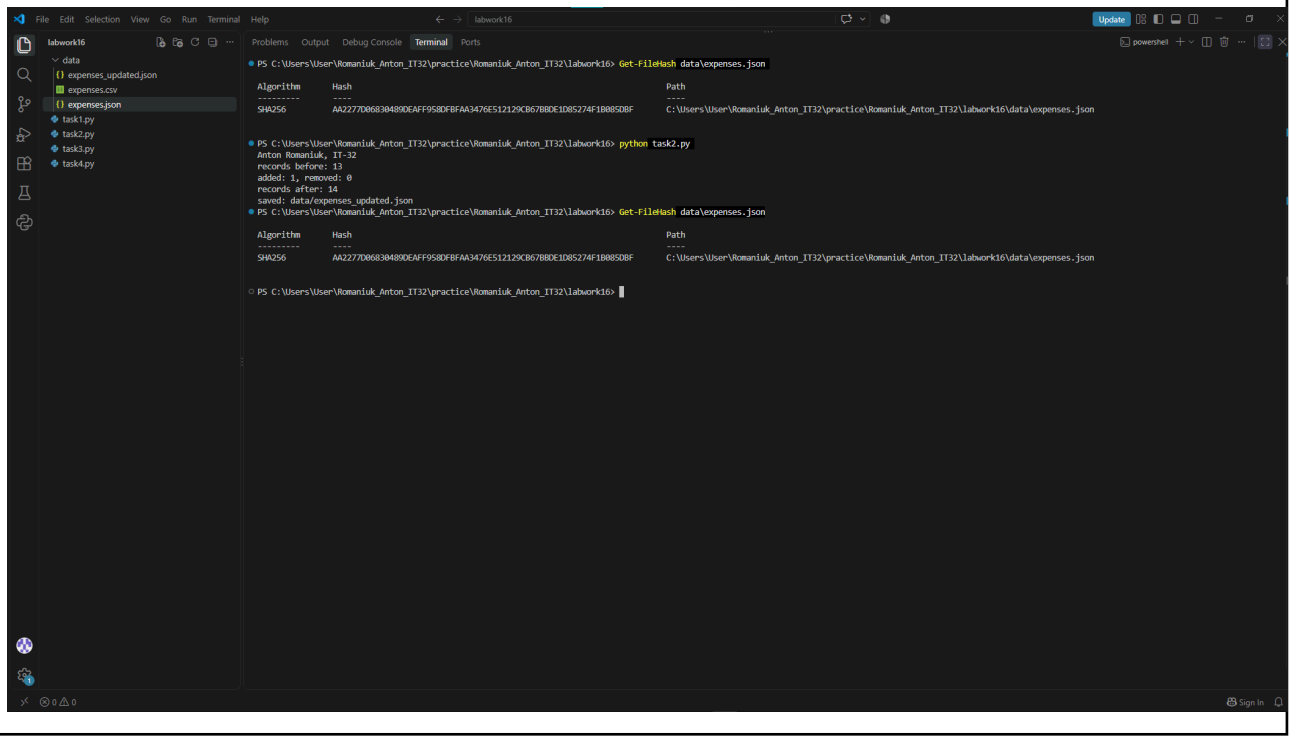
data/expenses.json у VS Code (суми записані числами, а не рядками; кирилиця не екранується):



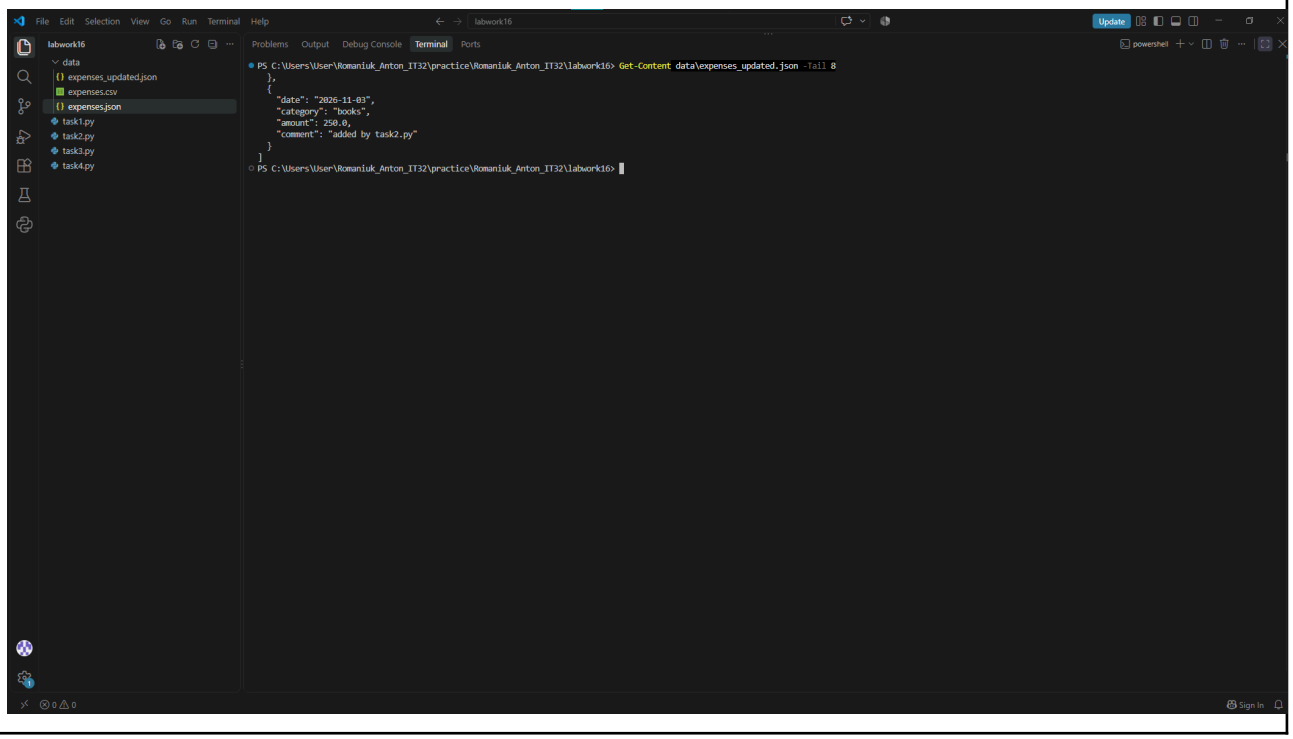
Завдання 2. Повний цикл:

Програма task2.py: читає data/expenses.json; додає новий запис із сьогоднішньою датою (date.today().isoformat()); видаляє всі записи із сумою менше 10; записує результат у data/expenses_updated.json і виводить, скільки записів додано та видалено. Вихідний файл не змінюється.

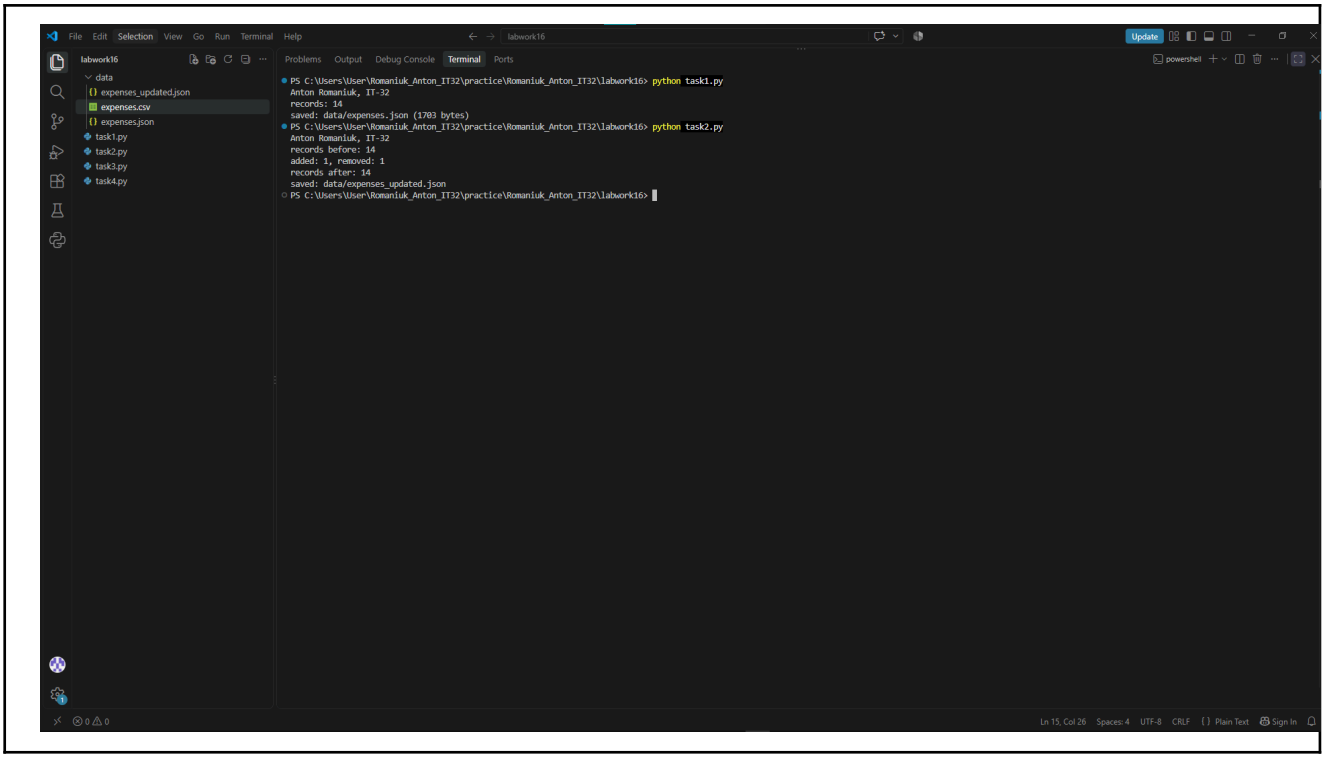
Get-FileHash data\expenses.json
python task2.py
Get-FileHash data\expenses.json



Get-Content data\expenses_updated.json -Tail 8



python task2.py



Завдання 3. Налаштування програми у settings.json

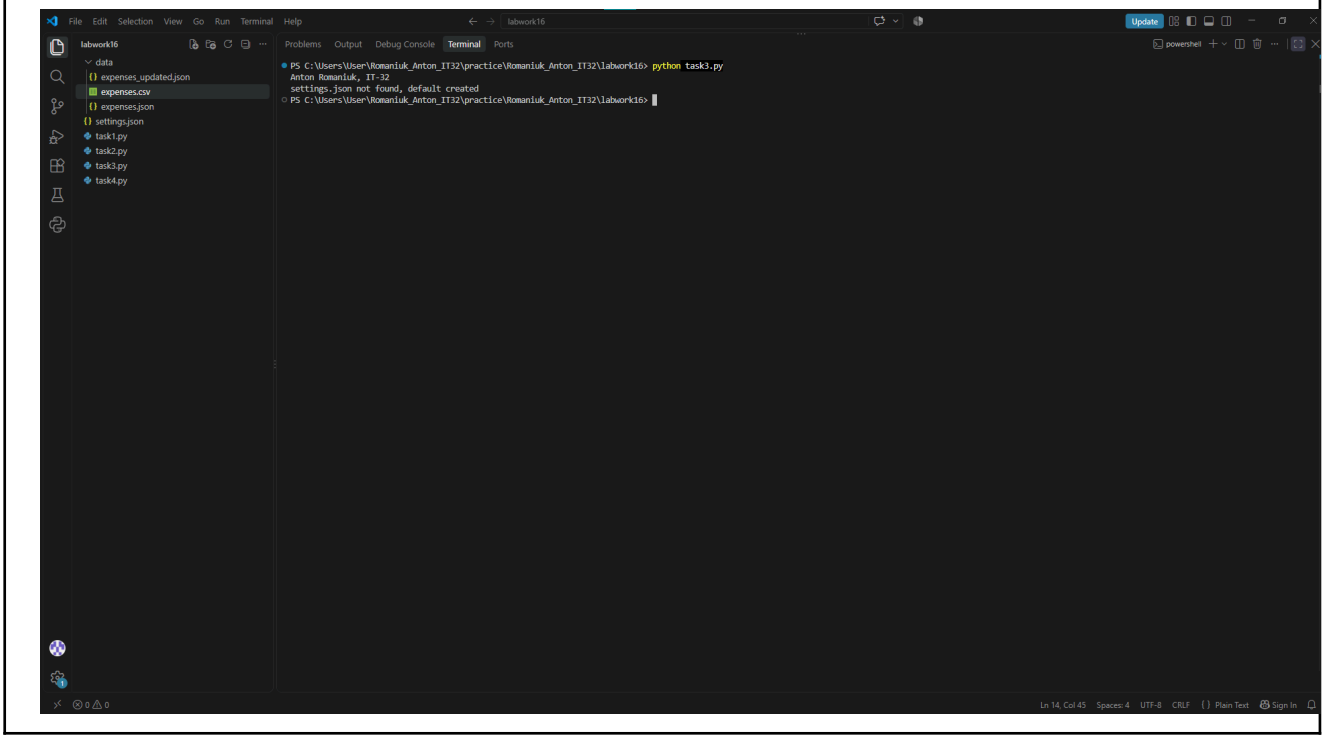
Програма task3.py зберігає мої налаштування у settings.json (owner, group, currency, monthly_limit, categories) та поводиться по-різному залежно від стану файлу:

• файлу немає — створює його зі значеннями за замовчуванням і повідомляє про це; • файл є — читає, виводить налаштування й порівнює загальну суму моїх витрат із monthly_limit;

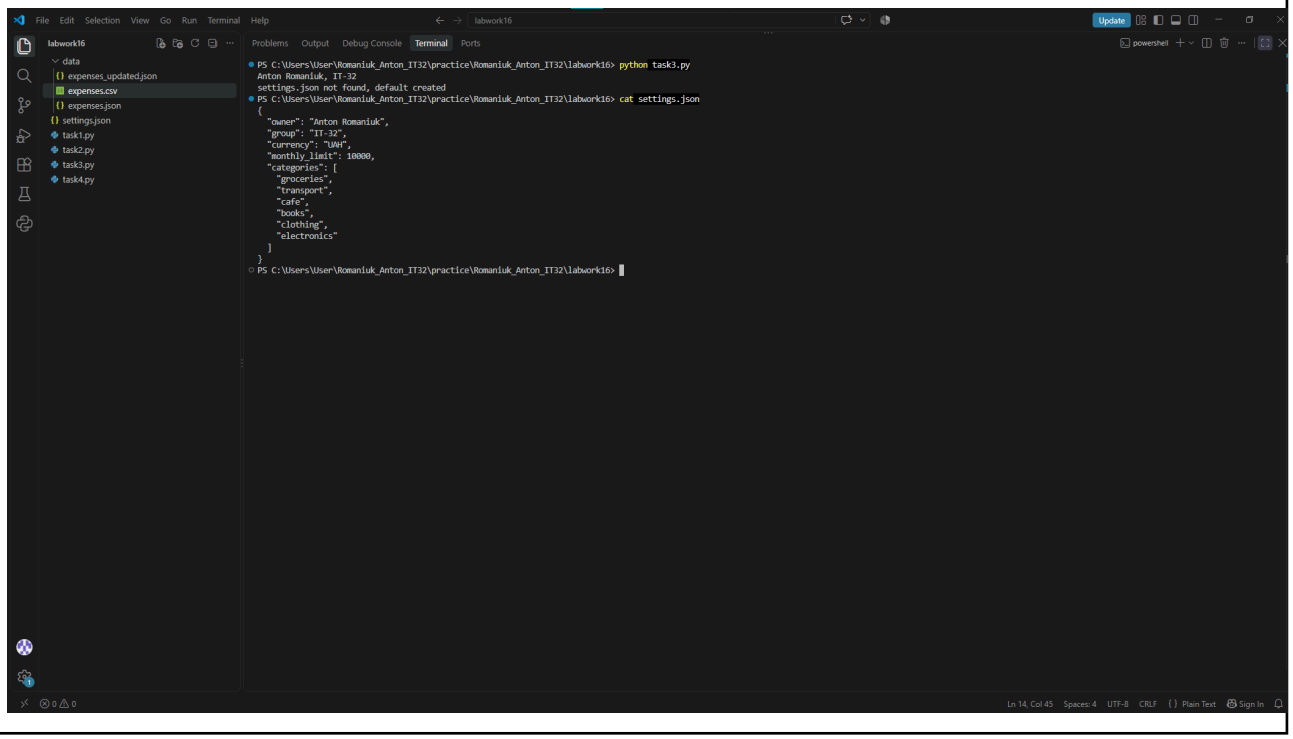
• файл пошкоджено — перехоплює json.JSONDecodeError, виводить номер рядка помилки в sys.stderr, завершується з кодом 1 і **не перезаписує** файл.

Запуск 1. Файлу немає:

python task3.py

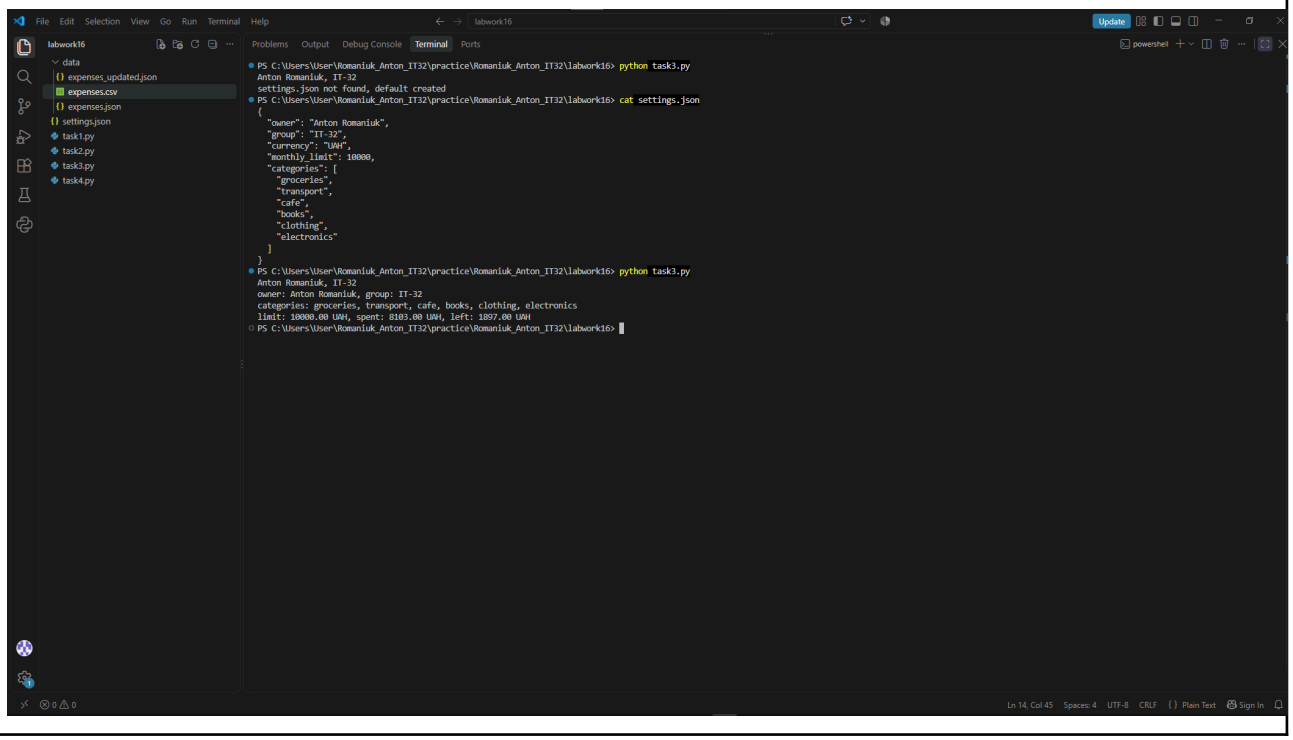


cat settings.json



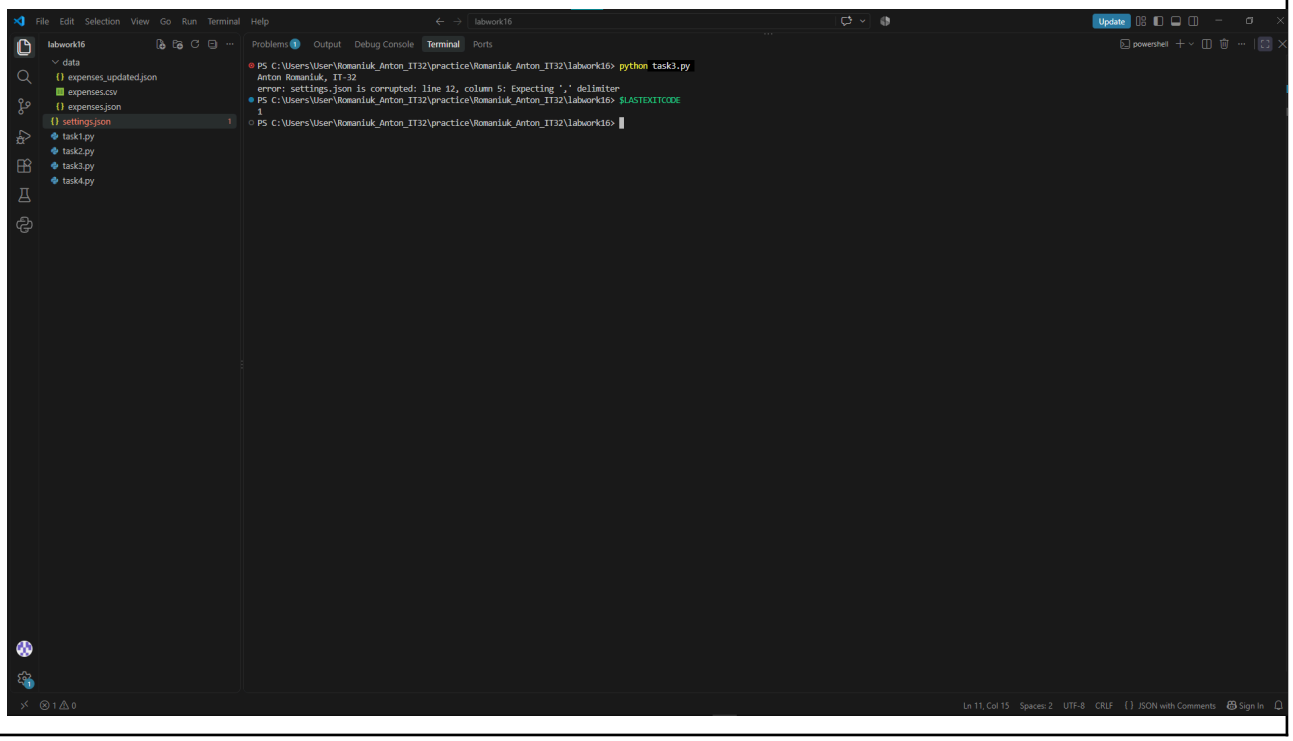
Запуск 2. Коректний файл:

python task3.py

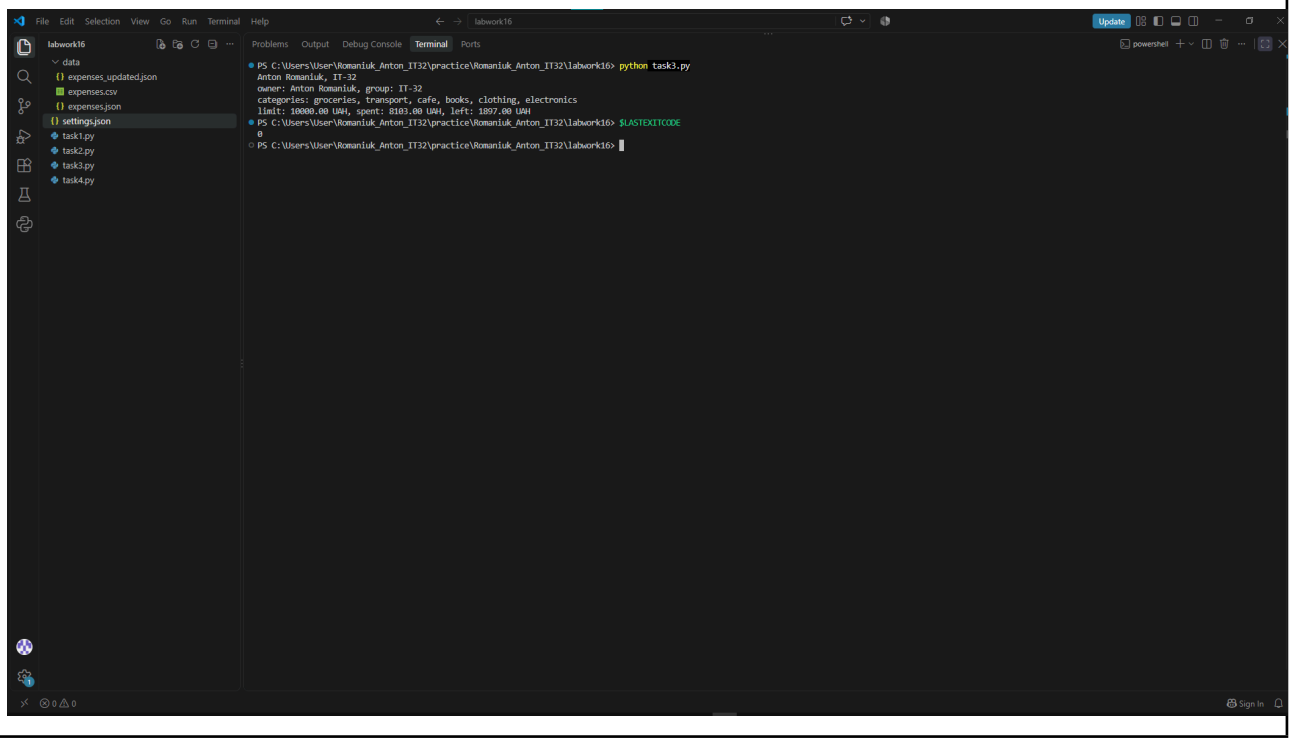


Запуск 3. Пошкоджений файл:

python task3.py
$LASTEXITCODE



python task3.py
$LASTEXITCODE



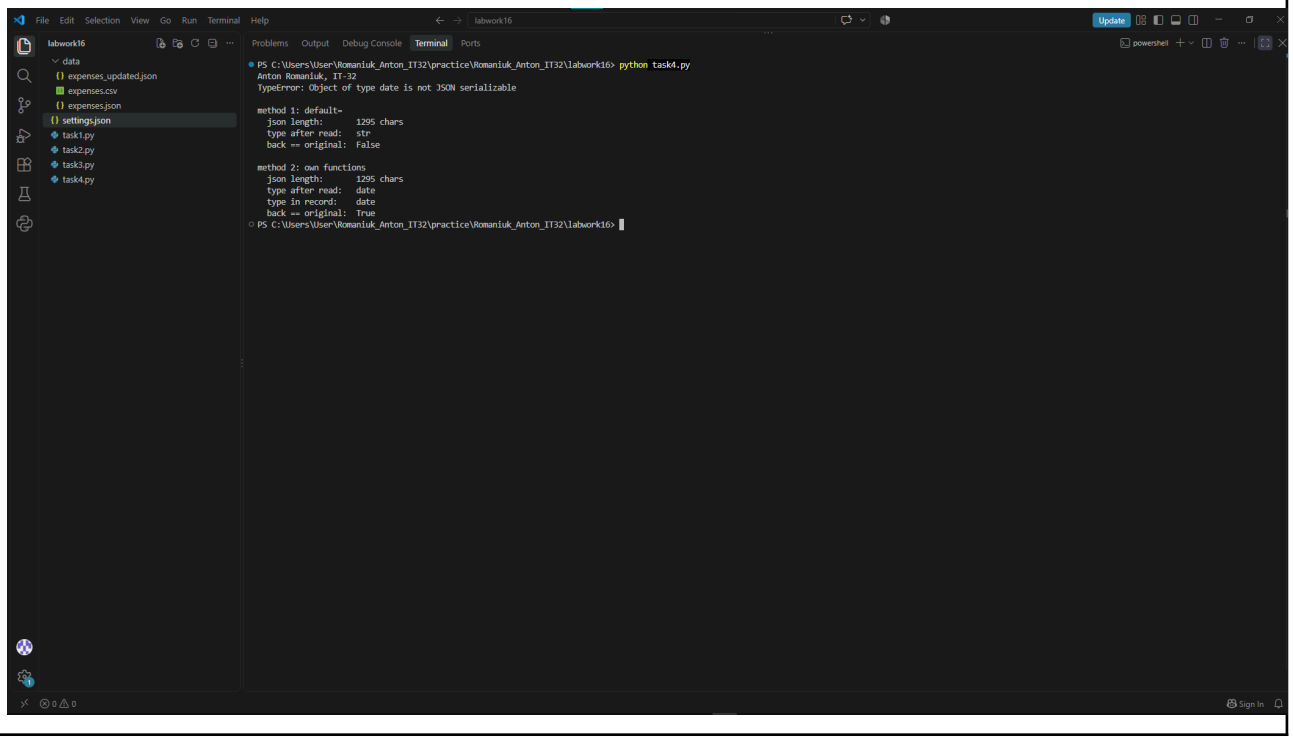
Завдання 4. Серіалізація дати

Програма task4.py будує словник із об'єктами date і намагається записати його в JSON.
Спочатку фіксується текст помилки TypeError, потім проблему розв'язано двома способами:

• **Спосіб 1:** параметр default= із функцією, що перетворює date на рядок ISO. Записати так можна, але після читання дати лишаються рядками;

• **Спосіб 2:** власні функції: dates_to_str перед записом замінює всі дати на рядки, а str_to_dates після читання повертає рядки формату YYYY-MM-DD назад на date.

python task4.py



Відповіді на контрольні запитання

1. Що таке серіалізація і навіщо вона потрібна?

Серіалізація — це перетворення даних програми (словників, списків, об'єктів), які існують у пам'яті, у послідовність байтів або текст, придатні для збереження чи передачі. Зворотний процес, відновлення даних з тексту, називається десеріалізацією (у json — load/loads). Вона потрібна, щоб:

• зберігати дані між запусками програми (налаштування, результати) у файлі; • передавати дані між програмами, мовами програмування та через мережу (API, відкриті дані на data.gov.ua);

• обмінюватися даними в зручному для людини й машини форматі, адже JSON читається і програмою, і в редакторі.

2. Які типи Python відповідають типам JSON? Що станеться з кортежем після запису й читання?

Відповідність: dict ↔ object, list та tuple ↔ array, str ↔ string, int/float ↔ number, True/False ↔ true/false, None ↔ null.

Кортеж у JSON не має власного типу: під час запису він стає масивом, а після читання повертається як **список** (list), тобто тип змінюється, і (1, 2) != [1, 2]. Також ключі словника в JSON завжди рядки (числовий ключ 1 стане "1"). Типи date, set, bytes та власні класи JSON не підтримує.

3. Чим json.dump відрізняється від json.dumps?

json.dump(obj, f) записує JSON у **файловий об'єкт** f (відкритий файл) і нічого не повертає. json.dumps(obj) повертає JSON як **рядок** (літера *s* — string), який можна вивести, надіслати мережею або записати самому. Так само json.load(f) читає з файлу, а json.loads(text) — з рядка.

4. Для чого параметри indent та ensure_ascii=False?

- indent задає відступи, і JSON записується в кілька рядків із відступами, тож файл легко читати й редагувати вручну (без нього все в одному рядку);
- ensure_ascii=False не екранує не-ASCII символи: кирилиця лишається читабельною, а не перетворюється на послідовності \uXXXX. Файл при цьому записують з encoding="utf-8".

5. Як обробити пошкоджений JSON-файл і чому його не можна мовчки перезаписати?

Читання обгортають у try/except json.JSONDecodeError. У винятку є номер рядка (lineno), стовпця (colno) і опис помилки (msg): їх виводять у sys.stderr і завершують програму з ненульовим кодом (sys.exit(1)).

Мовчки перезаписувати файл значеннями за замовчуванням не можна, бо користувач втратить свої дані, які, можливо, пошкоджені лише однією комою й легко виправляються. Мовчазна підміна ще й приховує проблему: програма «працює», але вже з чужими налаштуваннями.

6. Як серіалізувати тип, якого JSON не підтримує?

Є кілька способів:

- передати в json.dump/json.dumps параметр default= з функцією, що перетворює об'єкт на підтримуваний тип (для date — рядок ISO через isoformat(), для set — список); для невідомих типів вона має кидати TypeError;
- використати власний клас-кодувальник (підклас json.JSONEncoder);
- або заздалегідь перетворити дані власною функцією (як у моєму завданні 4), а після читання виконати зворотне перетворення (власна функція чи object_hook), інакше дані повернуться іншого типу (рядок замість date).